BEGINNER → ADVANCED

PYTHON PROGRAMMING

Python From Scratch + DSA

Understand Python's structure from the ground up, learn its built-in data structures, and solve 20+ problems progressing from beginner to advanced — all fully explained.

Part 1: **Core Python — syntax, data types, control flow, functions, OOP**

Part 2: **DSA notes with Python's built-in structures**

Part 3: **Solved problems: beginner → intermediate → advanced**

Every code block is runnable Python 3. Read top to bottom or jump via the contents.

Table of Contents

1. How Python Works & Program Structure

Python is a high-level, interpreted, dynamically-typed language. Understanding its structure removes most beginner confusion.

1.1 Key characteristics

- **Interpreted** — code runs line by line via the Python interpreter (CPython), no separate compile step.
- **Dynamically typed** — you don't declare types; a variable's type is inferred at runtime.
- **Indentation-based** — blocks are defined by indentation (4 spaces), not braces. This *is* the syntax.
- **Everything is an object** — numbers, functions, classes all are objects.

1.2 Your first program & anatomy

```
# 1. A comment starts with #
name = "Abrar"                # 2. variable assignment (no type needed)

def greet(person):            # 3. function definition
    """Return a greeting."""  #    docstring
    return f"Hello, {person}!" # 4. f-string formatting

if __name__ == "__main__":    # 5. entry point guard
    print(greet(name))        # 6. call + print -> Hello, Abrar!
```

The `if __name__ == "__main__":` guard ensures the code inside runs only when the file is executed directly, not when it's imported as a module elsewhere.

1.3 Indentation matters

```
# CORRECT
if True:
    print("inside the block")   # 4-space indent
    print("still inside")

# WRONG - IndentationError
if True:
print("no indent")              # will crash
```

Viva — Python basics

Q Is Python compiled or interpreted?

A Primarily interpreted — CPython compiles source to bytecode (.pyc) and then the interpreter (a virtual machine) executes it. There's no manual compile step like C.

Q What does "dynamically typed" mean?

A Variable types are determined at runtime, not declared. The same name can be rebound to a different type. Python is also strongly typed — it won't silently coerce incompatible types like "2"+2.

Q Why does indentation matter in Python?

A Indentation defines code blocks (loops, functions, conditionals) instead of braces. Inconsistent indentation raises an IndentationError — it's part of the grammar.

2. Variables & Data Types

2.1 Core built-in types

Type	Example	Mutable?
int	42	No (immutable)
float	3.14	No
bool	True / False	No
str	"hello"	No
list	[1, 2, 3]	Yes
tuple	(1, 2, 3)	No
set	{1, 2, 3}	Yes
dict	{"a": 1}	Yes
NoneType	None	—

```
x = 10               # int
pi = 3.14159         # float
flag = True          # bool
text = "Python"      # str
nums = [1, 2, 3]     # list
point = (4, 5)       # tuple
unique = {1, 2, 3}   # set
person = {"name": "Abrar", "age": 25}   # dict
nothing = None       # absence of value

print(type(x), type(text))   # <class 'int'> <class 'str'>
```

2.2 Mutable vs immutable

This concept trips up many beginners. **Immutable** objects (int, float, str, tuple) cannot be changed in place — operations create new objects. **Mutable** objects (list, set, dict) can be modified.

```
s = "cat"
s[0] = "b"           # TypeError! strings are immutable

lst = [1, 2, 3]
lst[0] = 99          # OK - lists are mutable -> [99, 2, 3]
```

2.3 Type conversion

```
int("42")     # 42     (str -> int)
str(42)       # "42"   (int -> str)
float("3.5")  # 3.5
list("abc")   # ['a', 'b', 'c']
```

Viva — Data types

Q Difference between a list and a tuple?

A Lists are mutable (can add/remove/change items) and use []; tuples are immutable and use (). Tuples are hashable so they can be dict keys or set elements; lists cannot.

Q Mutable vs immutable — why does it matter?

A Immutable objects can't change in place, so they're safe to share and hashable. Mutable objects can cause surprising bugs when passed around (aliasing) and can't be used as dict keys.

Q What is None?

A A special singleton representing "no value" / absence of a result. Functions with no explicit return return None. Compare with `is None`, not `== None`.

Q is vs == ?

A `==` checks value equality; `is` checks identity (same object in memory). Use `is` for None/True/False, `==` for value comparisons.

3. Operators

Category	Operators
Arithmetic	+ - * / (float div) // (floor div) % (modulo) ** (power)
Comparison	== != < > <= >=
Logical	and or not
Assignment	= += -= *= //= **=
Membership	in not in
Identity	is is not

```
print(7 // 2)    # 3   floor division
print(7 % 2)     # 1   remainder
print(2 ** 10)   # 1024 power
print("a" in "cat")     # True  membership
print(5 > 3 and 2 < 1)  # False logical
```

4. Control Flow

4.1 Conditionals

```python
score = 82
if score >= 90:
    grade = "A"
elif score >= 80:
    grade = "B"
else:
    grade = "C"
print(grade)     # B

# Ternary (one-liner)
status = "pass" if score >= 40 else "fail"
```

4.2 Loops

```python
# for over a range
for i in range(3):        # 0, 1, 2
    print(i)

# for over a collection
for fruit in ["apple", "banana"]:
    print(fruit)

# while
n = 5
while n > 0:
    print(n)
    n -= 1

# loop control
for i in range(10):
    if i == 3:
        continue          # skip this iteration
    if i == 6:
        break             # exit the loop
    print(i)

# enumerate + zip (very common)
names = ["a", "b"]
for idx, name in enumerate(names):     # 0 a, 1 b
    print(idx, name)
for x, y in zip([1,2], [3,4]):         # (1,3), (2,4)
    print(x, y)
```

Viva — Control flow

Q break vs continue?

A `break` exits the entire loop immediately; `continue` skips the rest of the current iteration and moves to the next.

Q What does the for-else construct do?

A The else block after a for/while runs only if the loop completed without hitting break — useful for search loops ("not found" cases).

Q What do enumerate and zip do?

A enumerate yields (index, value) pairs while iterating; zip pairs elements from multiple iterables together, stopping at the shortest.

5. Built-in Data Structures

5.1 List — ordered, mutable

```
nums = [3, 1, 2]
nums.append(4)          # [3,1,2,4]
nums.insert(0, 9)       # [9,3,1,2,4]
nums.remove(1)          # removes first 1
nums.pop()              # removes & returns last
nums.sort()             # in-place sort
print(nums[0], nums[-1])   # first, last
print(nums[1:3])        # slicing
```

5.2 Tuple — ordered, immutable

```
point = (3, 4)
x, y = point            # unpacking
# point[0] = 5          # error: immutable
```

5.3 Set — unordered, unique

```
s = {1, 2, 2, 3}        # {1,2,3} - duplicates removed
s.add(4)
print(3 in s)           # True - O(1) membership
a, b = {1,2,3}, {2,3,4}
print(a & b)            # {2,3} intersection
print(a | b)            # {1,2,3,4} union
print(a - b)            # {1} difference
```

5.4 Dict — key-value pairs

```
person = {"name": "Abrar", "age": 25}
person["role"] = "SQA"          # add/update
print(person.get("email", "n/a"))   # safe access with default
for key, val in person.items():
    print(key, val)
print("age" in person)          # True - O(1) key lookup
```

Viva — Data structures

Q When do you use a set vs a list?

A A set for uniqueness and fast O(1) membership tests and set math (union/intersection); a list when order and duplicates matter or you need indexing.

Q Why is dict lookup fast?

A A dict is a hash table, so key lookup/insert/delete are average O(1). Since Python 3.7 it also preserves insertion order.

Q Difference between dict.get(k) and dict[k]?

A `dict[k]` raises KeyError if the key is missing; `dict.get(k, default)` returns a default (None if unspecified) instead of raising.

Q What is list slicing?

A `lst[start:stop:step]` returns a new sub-list; omitting parts uses defaults, and `lst[::-1]` reverses a list.

6. Functions

```python
def add(a, b=0):                # b has a default value
    return a + b

def stats(*args):               # *args -> tuple of positional args
    return sum(args), len(args)

def config(**kwargs):           # **kwargs -> dict of keyword args
    return kwargs

print(add(3))          # 3
print(add(3, 4))       # 7
print(stats(1,2,3))    # (6, 3)
print(config(env="prod", debug=True))  # {'env': 'prod', 'debug': True}
```

Scope (LEGB rule): Python resolves names in order Local → Enclosing → Global → Built-in. Use `global` / `nonlocal` to modify outer-scope variables (rarely needed).

Mutable default argument trap: `def f(x, items=[])` reuses the same list across calls. Use `items=None` then `items = items or []` inside.

Viva — Functions

Q What are *args and **kwargs?

A `*args` collects extra positional arguments into a tuple; `**kwargs` collects extra keyword arguments into a dict. They let functions accept a variable number of arguments.

Q Explain the LEGB scope rule.

A Name lookup goes Local → Enclosing (outer functions) → Global (module) → Built-in. The first match wins.

Q Why avoid mutable default arguments?

A Defaults are evaluated once at definition, so a shared list/dict default persists across calls, causing surprising accumulation. Use None as the default and create the object inside.

Q Are Python arguments passed by value or reference?

A "Pass by object reference" — the reference is passed by value. Rebinding a parameter doesn't affect the caller, but mutating a mutable argument (e.g., appending to a list) does.

7. Strings & Formatting

```
s = "Hello, World"
print(len(s))            # 12
print(s.upper())         # HELLO, WORLD
print(s.lower())
print(s.replace("World", "Python"))
print(s.split(", "))     # ['Hello', 'World']
print("-".join(["a","b","c"]))  # a-b-c
print(s[0:5])            # Hello (slice)
print(s[::-1])           # reversed
print("  trim  ".strip())    # 'trim'

# f-strings (preferred formatting)
name, score = "Abrar", 95
print(f"{name} scored {score}%")
print(f"{3.14159:.2f}")  # 3.14  (2 decimals)
print(f"{42:05d}")       # 00042 (zero-padded)
```

Viva — Strings

Q Why are f-strings preferred?

A They're concise, readable, and fast — expressions embed directly inside {} and support formatting specs like :.2f, unlike older % or .format() styles.

Q How do you reverse a string?

A Slice with a step of -1: `s[::-1]`.

8. Comprehensions & Lambdas

Pythonic one-liners that replace verbose loops — heavily used and often asked about.

```python
# List comprehension
squares = [x*x for x in range(5)]           # [0,1,4,9,16]
evens   = [x for x in range(10) if x % 2==0]# [0,2,4,6,8]

# Dict & set comprehensions
sq_map = {x: x*x for x in range(4)}         # {0:0,1:1,2:4,3:9}
unique = {c for c in "banana"}              # {'b','a','n'}

# Nested
matrix = [[1,2],[3,4]]
flat = [n for row in matrix for n in row]   # [1,2,3,4]

# lambda + map/filter/sorted
nums = [3,1,2]
doubled = list(map(lambda x: x*2, nums))    # [6,2,4]
big     = list(filter(lambda x: x>1, nums)) # [3,2]
by_desc = sorted(nums, key=lambda x: -x)    # [3,2,1]
```

Viva — Comprehensions

Q What is a list comprehension and why use it?

A A concise syntax to build a list from an iterable with an optional condition: `[expr for item in iterable if cond]`. It's more readable and usually faster than an equivalent loop with append.

Q What is a lambda?

A An anonymous, single-expression function. Useful as a short throwaway callback, e.g., a sort key or a map/filter function.

Q Generator expression vs list comprehension?

A A generator (parentheses instead of brackets) yields items lazily one at a time, using O(1) memory — ideal for large/streamed data — while a list comprehension builds the whole list in memory.

9. Object-Oriented Programming

```python
class Animal:                          # base class
    def __init__(self, name):          # constructor
        self.name = name               # instance attribute
    def speak(self):                   # method
        return "..."
    def __str__(self):                 # string representation
        return f"Animal({self.name})"

class Dog(Animal):                     # inheritance
    def speak(self):                   # polymorphism (override)
        return "Woof!"

class Cat(Animal):
    def speak(self):
        return "Meow!"

for a in [Dog("Rex"), Cat("Tom")]:
    print(a.name, a.speak())           # Rex Woof! / Tom Meow!
```

9.1 Four OOP pillars

- **Encapsulation** — bundle data + methods; hide internals (prefix `_` convention, `__` name-mangling).
- **Inheritance** — a subclass reuses/extends a base class.
- **Polymorphism** — same method name, different behavior per class (like `speak()`).
- **Abstraction** — expose essentials, hide complexity (abstract base classes).

Viva — OOP

Q Name the four OOP principles.

A Encapsulation (bundling & hiding data), Inheritance (reuse via subclassing), Polymorphism (same interface, different behavior), and Abstraction (hiding complexity behind a simple interface).

Q What is self?

A The reference to the current instance, passed automatically as the first parameter of instance methods, used to access that object's attributes and methods.

Q Difference between __init__ and __str__?

A `__init__` is the constructor that initializes a new object's attributes; `__str__` defines the human-readable string shown by print()/str().

Q Instance vs class vs static methods?

A Instance methods take self and act on an object; class methods take cls (@classmethod) and act on the class; static methods (@staticmethod) take neither and are just namespaced utility functions.

10. Modules, Files & Exceptions

10.1 Modules & imports

```
import math
from collections import Counter, deque
print(math.sqrt(16))          # 4.0
print(Counter("banana"))      # Counter({'a':3,'n':2,'b':1})
```

10.2 File handling

```
# 'with' auto-closes the file even if an error occurs
with open("data.txt", "w") as f:
    f.write("line 1\nline 2\n")

with open("data.txt") as f:
    for line in f:
        print(line.strip())
```

10.3 Exceptions

```
try:
    result = 10 / 0
except ZeroDivisionError as e:
    print("Cannot divide by zero:", e)
except (ValueError, TypeError):
    print("bad input")
else:
    print("ran only if no exception")
finally:
    print("always runs (cleanup)")
```

Viva — Modules / Files / Exceptions

Q Why use "with open(...)"?

A It's a context manager that guarantees the file is closed automatically when the block exits, even on error — preventing resource leaks.

Q Purpose of try/except/finally?

A try runs risky code, except handles specific errors gracefully, else runs if no error occurred, and finally always runs for cleanup (closing resources) regardless of outcome.

Q Difference between a module and a package?

A A module is a single .py file; a package is a directory of modules (traditionally with an __init__.py) that groups related modules together.

11. DSA Notes with Python Built-ins

Python ships with high-quality data structures. Knowing which built-in maps to which abstract structure saves you from reinventing them.

Abstract structure	Python built-in	Key ops & complexity
Dynamic array	`list`	index O(1), append O(1)*, insert/delete mid O(n)
Stack (LIFO)	`list`	append / pop O(1)
Queue / Deque	`collections.deque`	append / popleft O(1)
Hash map	`dict`	get/set/del avg O(1)
Hash set	`set`	add / in avg O(1)
Min-heap / PQ	`heapq`	push / pop O(log n), peek O(1)
Counter (freq map)	`collections.Counter`	counting O(n)
Ordered default map	`collections.defaultdict`	auto-init missing keys

Essential imports for interviews

```python
from collections import deque, Counter, defaultdict
import heapq
from functools import lru_cache      # instant memoization decorator

# defaultdict: no KeyError, auto-creates default
graph = defaultdict(list)
graph["A"].append("B")               # no need to check if "A" exists

# Counter: frequency in one line
freq = Counter("mississippi")        # {'i':4,'s':4,'p':2,'m':1}
print(freq.most_common(2))           # [('i',4),('s',4)]

# lru_cache: memoize a recursive function
@lru_cache(maxsize=None)
def fib(n):
    return n if n < 2 else fib(n-1) + fib(n-2)
```

Interview shortcut: `@lru_cache` converts an exponential recursive solution into a linear memoized one with a single decorator — great for DP problems under time pressure.

Viva — Python DSA built-ins

Q Which structure for a queue in Python and why not a list?

A `collections.deque` — it gives O(1) popleft, whereas a list's pop(0) is O(n) due to shifting.

Q What does defaultdict solve?

A It removes repetitive "if key not in dict" checks by auto-initializing missing keys with a factory (list, int, set), which simplifies building graphs and frequency maps.

Q What does @lru_cache do?

A It's a decorator that memoizes a function's results by arguments, turning repeated calls into cache hits — an instant way to add DP-style memoization.

12. Solved Problems — Beginner

P1. Sum of a list

```python
def total(nums):
    s = 0
    for n in nums:
        s += n
    return s
# total([1,2,3,4]) -> 10   (built-in: sum(nums))
```

Explanation: iterate once, accumulate. O(n) time, O(1) space.

P2. Find max without max()

```python
def maximum(nums):
    best = nums[0]
    for n in nums[1:]:
        if n > best:
            best = n
    return best
```

Explanation: track the largest seen so far. O(n).

P3. Reverse a string

```python
def reverse(s):
    return s[::-1]           # slicing trick
# manual: ''.join(reversed(s))
```

P4. Count vowels

```python
def count_vowels(s):
    return sum(1 for ch in s.lower() if ch in "aeiou")
```

Explanation: generator expression counts matches in one pass. O(n).

P5. FizzBuzz

```python
for i in range(1, 16):
    if i % 15 == 0: print("FizzBuzz")
    elif i % 3 == 0: print("Fizz")
    elif i % 5 == 0: print("Buzz")
    else: print(i)
```

Explanation: check the most restrictive condition (divisible by both) first.

P6. Check palindrome

```python
def is_palindrome(s):
    s = s.lower().replace(" ", "")
    return s == s[::-1]
```

13. Solved Problems — Intermediate

P7. Two Sum (hash map)

```python
def two_sum(nums, target):
    seen = {}
    for i, x in enumerate(nums):
        if target - x in seen:
            return [seen[target - x], i]
        seen[x] = i
    return []
# two_sum([2,7,11,15], 9) -> [0,1]
```

Explanation: store each number's index; for each element check if its complement was seen. O(n) instead of O(n²).

P8. First non-repeating character

```python
from collections import Counter
def first_unique(s):
    counts = Counter(s)
    for ch in s:
        if counts[ch] == 1:
            return ch
    return None
```

Explanation: count frequencies (O(n)), then return the first with count 1.

P9. Group anagrams

```python
from collections import defaultdict
def group_anagrams(words):
    groups = defaultdict(list)
    for w in words:
        key = "".join(sorted(w))     # anagrams share sorted letters
        groups[key].append(w)
    return list(groups.values())
# ["eat","tea","tan","ate"] -> [["eat","tea","ate"],["tan"]]
```

Explanation: the sorted string is a canonical key that all anagrams share.

P10. Fibonacci (memoized)

```python
from functools import lru_cache
@lru_cache(maxsize=None)
def fib(n):
    return n if n < 2 else fib(n-1) + fib(n-2)
```

Explanation: caching subresults turns O(2^n) into O(n).

P11. Valid parentheses (stack)

```python
def valid(s):
    pairs = {')':'(', ']':'[', '}':'{'}
    stack = []
    for ch in s:
        if ch in "([{":
            stack.append(ch)
        elif not stack or stack.pop() != pairs[ch]:
            return False
    return not stack
```

P12. Merge two sorted lists

```python
def merge(a, b):
    res, i, j = [], 0, 0
    while i < len(a) and j < len(b):
        if a[i] <= b[j]:
            res.append(a[i]); i += 1
        else:
            res.append(b[j]); j += 1
    res.extend(a[i:]); res.extend(b[j:])
    return res
```

Explanation: two pointers walk both sorted lists, always taking the smaller head. O(n+m).

14. Solved Problems — Advanced

P13. Longest substring without repeating characters (sliding window)

```python
def longest_unique(s):
    last = {}                 # char -> last index seen
    start = best = 0
    for i, ch in enumerate(s):
        if ch in last and last[ch] >= start:
            start = last[ch] + 1     # shrink window past the repeat
        last[ch] = i
        best = max(best, i - start + 1)
    return best
# "abcabcbb" -> 3 ("abc")
```

Explanation: a window [start..i] holds unique chars; when a repeat appears, jump start past its previous position. O(n).

P14. Binary search

```python
def bsearch(arr, target):
    lo, hi = 0, len(arr) - 1
    while lo <= hi:
        mid = (lo + hi) // 2
        if arr[mid] == target: return mid
        elif arr[mid] < target: lo = mid + 1
        else: hi = mid - 1
    return -1
```

P15. Quicksort

```python
def quicksort(a):
    if len(a) <= 1: return a
    pivot = a[len(a)//2]
    return (quicksort([x for x in a if x < pivot])
            + [x for x in a if x == pivot]
            + quicksort([x for x in a if x > pivot]))
```

P16. BFS shortest path in a grid

```python
from collections import deque
def shortest_path(grid, start, goal):
    rows, cols = len(grid), len(grid[0])
    q = deque([(start, 0)])
    seen = {start}
    while q:
        (r, c), dist = q.popleft()
        if (r, c) == goal:
            return dist
        for dr, dc in [(-1,0),(1,0),(0,-1),(0,1)]:
            nr, nc = r+dr, c+dc
            if 0 <= nr < rows and 0 <= nc < cols \
               and grid[nr][nc] == 0 and (nr,nc) not in seen:
                seen.add((nr,nc))
                q.append(((nr,nc), dist+1))
    return -1
```

Explanation: BFS explores cells level by level, so the first time it reaches the goal is the shortest path in an unweighted grid.

P17. Coin change (DP)

```python
def coin_change(coins, amount):
    dp = [0] + [float('inf')] * amount
    for a in range(1, amount + 1):
        for c in coins:
            if c <= a:
                dp[a] = min(dp[a], dp[a-c] + 1)
    return dp[amount] if dp[amount] != float('inf') else -1
```

Explanation: dp[a] = fewest coins to make amount a; build up from 0. O(amount × coins).

P18. Top-K frequent elements (heap)

```python
import heapq
from collections import Counter
def top_k(nums, k):
    counts = Counter(nums)
    return heapq.nlargest(k, counts.keys(), key=counts.get)
# top_k([1,1,1,2,2,3], 2) -> [1,2]
```

Explanation: count frequencies, then use a heap to pull the k highest in O(n log k).

15. Python Interview Quick Q&A

Q What is the GIL?

A The Global Interpreter Lock lets only one thread execute Python bytecode at a time in CPython, so threads don't speed up CPU-bound work (use multiprocessing for that). I/O-bound tasks still benefit from threads since the GIL is released during I/O.

Q List vs tuple vs set vs dict — one line each?

A List: ordered, mutable, allows duplicates. Tuple: ordered, immutable. Set: unordered, unique. Dict: key-value pairs with O(1) lookup.

Q Shallow vs deep copy?

A A shallow copy copies the outer object but shares nested references; a deep copy (copy.deepcopy) recursively copies everything, so nested objects are independent.

Q What is a decorator?

A A function that wraps another function to add behavior (logging, caching, timing) without modifying it, applied with @syntax.

Q What is a generator?

A A function using yield that produces values lazily one at a time, keeping O(1) memory — ideal for large or streaming sequences.

Q == vs is?

A == compares values; is compares object identity (same memory location).

Q How does Python manage memory?

A Via reference counting plus a cyclic garbage collector for reference cycles; objects are freed when their reference count drops to zero.

Q What are list/dict/set comprehensions good for?

A Concise, readable, and often faster construction of collections from iterables with optional filtering/transforming.

Practice plan: re-implement each solved problem from scratch without looking, then explain the time/space complexity aloud. If you can code it and narrate the trade-offs, you're interview-ready.